Homework — 2.1.1 Thinking Abstractly — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. (a) [2] 1 mark: nodes represent junctions / road intersections / locations (e.g. delivery addresses). 1 mark: edge weights represent the **cost of travel** between connected nodes — for the *quickest* route this should be **time** (or distance if a speed is assumed), so a shortest-path algorithm can work on the weights.

(b) [2] 1 mark for a plausible discarded detail, e.g. street names / road surface / building colours / the scenery. 1 mark for justification: such detail has no effect on how long it takes to travel a road, so it is irrelevant to choosing the fastest route and would only add complexity. Must justify for the second mark.

2. (a) [2] 1 mark each, any two, e.g.: a "crash" has **no real consequences/danger**; motion and G-forces are absent or only approximated; the world outside the cockpit is **computer-generated graphics**, not real terrain/weather; weather and equipment failures are generated on demand by software; no real fuel is used / no wear on a real aircraft.

(b) [1] The model **keeps the details that matter** for training (the controls, instrument readings and how the aircraft responds) while removing irrelevant or dangerous ones — the abstraction only needs to match reality for its **purpose**, so pilots practise the same decisions and skills safely and cheaply.

3. [2] 1 mark: the system **hides/removes** the complex underlying detail (pressure, humidity, grid model) and presents only a simple summary (an icon) — a higher-level abstraction for the user. 1 mark: this is appropriate because the public only need to know whether to expect sun/rain, not the raw meteorological data, so the simpler model is easier to understand and act on.

4. [2] 1 mark: it is wrong because splitting a problem into smaller parts (move-checker, display, save-game) is **decomposition**, not abstraction; abstraction *removes/hides* unnecessary detail rather than breaking the problem up. 1 mark: name the skill correctly as **decomposition** (thinking procedurally). *(Common trap from the spec.)*

5. [3] 1 mark for two sensible *keeps*, e.g. visitor's place/position in the queue, arrival/join time, party size, ride eligibility (height) — any two. 1 mark for a discarded detail **with justification**, e.g. the visitor's favourite colour / what they had for lunch — it plays no part in queue position or wait time, so it is irrelevant to this model. 1 mark for the reason: removing irrelevant detail keeps the model simpler, so it is faster to build, easier to understand and maintain while still solving the queue problem. Must apply to *this* scenario.

Part B (6 marks)

6. [2] 1 mark: the PC **holds the address of the next instruction** to be fetched. 1 mark: this address is copied to the MAR at the start of fetch, and the PC is then **incremented** (or updated by a jump) so the cycle moves through the program in order.

7. [2] 1 mark + 1 mark, any one developed advantage: SSDs have **no moving parts**, so they are faster (lower access/seek time) **[1]**; also more durable/shock-resistant, silent and lower power **[1]**. *(Accept any one valid advantage stated clearly for both marks.)*

8. [2] 1 mark: pipelining is fetching/decoding/executing **overlapped** — while one instruction is executed, the next is decoded and the one after is fetched. 1 mark: benefit — more instructions are completed per unit time / better use of the CPU / higher throughput.

Total: 14 + 6 = 20 marks